VS Code MSSQL Updates: Design Document Outline

Working inventory for `dev/query` across `vscode-mssql`, `sqltoolsservice`, and `perftest`

July 7, 2026

Abstract

This paper should become the consolidated design and usage document for the `dev/query` branch family. It should explain how the STS2 refactor, SQL Data Plane, metadata substrate, Query Studio, native language service, AI completions, Debug Console, central observability, and perftest CLI fit together. The immediate goal of this draft is an outline plus a source-material index: each section names the existing Markdown, TeX diagrams, code areas, and screenshot folders that should be copied in, edited, or used as figures later.

Contents

1 Scope and Current Branch Inventory

This document is scoped to the current `dev/query` branches under `C:/repos/test`. The inventory below is based on diffs against `main` and a targeted read of the design packs, Markdown docs, TeX visual packs, and screenshot folders in the sibling repositories.

Repository	Diff size vs. `main`	High-signal change areas
`vscode-mssql`	301 files, 111262 insertions, 33 deletions	SQL Data Plane and STS2 binding; Query Studio custom editor; MetadataStore and cache; native TypeScript T-SQL language service and scripting; AI completions and SDK model providers; Debug Console; central upload; Object Explorer v2; shared observability contracts and perf hooks.
`sqltoolsservice`	298 files, 28783 insertions, 82 deletions	In-process STS2 service core, runtime, multiplexer, hosting, drivers, journaling, replay, redaction, observability sinks, generated docs, review pack, verification scripts, scenarios, and tests.
`perftest`	54 files, 10815 insertions, 33 deletions	Central observability contract/store, `perftest push`, central report, head-to-head comparison, Query Studio scenario support, Grafana dashboard, generated event docs, CLI docs, and central-store tests.

1.1 Minimum Topic Set

The first complete draft should cover these topics explicitly:

- STS2 refactor and the service-side v2 contract.
- VS Code metadata service, MetadataStore, server catalog, cache, and metadata consumers.
- New native TypeScript language service and scripting/definition support.
- Query Studio and the SQL Data Plane endpoint adapters.
- AI completions, schema-context construction, SDK plugins, and replay.
- Debug Console, session diagnostics, feature capture, and central observability.
- perftest CLI, scenario authoring, local reports, head-to-head comparisons, central push, and how to operate it.

1.2 Additional Inventory to Decide In or Out

The branch also contains adjacent work that may deserve sections or appendices, depending on what survives review:

- Object Explorer v2 preview over the MetadataStore.

- Metadata persistent cache and offline mode.
- Central observability upload from the product and central read-provider plans.
- Query Studio Replay Lab and shared feature-capture framework.
- Inline Completion Debug viewer and completion replay matrix.
- MCP bridge and other endpoint-adapter experiments, if they remain in scope.
- Data API Builder, schema/table designer touch points, and other extension surfaces if they changed materially on this branch.

2 Proposed Paper Structure

2.1 1. Executive Summary

Explain the integrated story in two pages: the branch moves SQL execution onto STS2 through a SQL Data Plane, builds Query Studio as the first major product consumer, centralizes metadata for language features and OE v2, exposes diagnostics through the Debug Console, and uses perftest/central observability to measure and share evidence. This section should also state what is experimental, what is preview-ready, and what might be removed.

Source material: `../coding-docs/remaining_tasks.md`, `../coding-docs/settings.md`, `../coding-docs/observability-docs/07-phase2-query-studio-branch-guide.md`

2.2 2. System Architecture Overview

Give the reader one mental model before diving into components:

- VS Code UI surfaces and controllers.
- SQL Data Plane domain API and backend bindings.
- STS2 service-side v2 core and legacy side-by-side hosting.
- MetadataStore as the shared substrate for Query Studio, language service, AI completions, and OE v2.
- Diagnostics, replay, perftest, and central store as a shared evidence pipeline.

Candidate figure: a synthesized architecture diagram using the visual language from the existing STS2, metadata, and central observability TeX packs.

3 STS2 Refactor

3.1 Purpose and Boundaries

Document STS2 as a minimal in-process v2 subsystem inside SqlToolsService: connectivity, query execution, diagnostics, journaling, replay, and redacted export. State the non-goals clearly: no v2.0 language service, Object Explorer, scripting, edit data, notebooks, profiling, server-side batch splitting, or random-access grid cache.

3.2 Implementation Design

Cover the project map and call graph:

- `Contracts`: wire constants, methods, stable errors, defaults.
- `Core`: pure reducer, immutable state, connection/query state machines.
- `Runtime`: coordinator pump, write-ahead journal, effects, replay, redaction, observability sinks.
- `Multiplexer`: one stdio channel, legacy and v2 routing, ID rewrite, lifecycle mirroring.
- `Hosting` and `Bootstrap`: the thin seam into the existing service process.
- `Drivers.SqlClient`, `Drivers.Sqlite`, and `Testing`: production driver, portable real-I/O driver, fake driver, YAML scenarios, simulator, invariant checker.

3.3 Journaling, Replay, Redaction, and Observability

This should be one of the detailed design chapters. Explain envelopes, sequence order, write-ahead journaling, strict replay, capture elision, secret side tables, `IEnvelopeSink`, live tail, metrics sink, and export bundles.

3.4 Validation and Remaining Gates

Summarize generated docs, scenarios, invariant tests, replay tests, mutation tests, SQL Server container coverage, and the M7 preview tag / human-review gate.

Primary sources: `../sqltoolsservice/docs/sts2/SPEC.md`, `../sqltoolsservice/docs/sts2/CONTRACT.md`, `../sqltoolsservice/refactor_docs/what_was_built.md`, `../sqltoolsservice/refactor_docs/STS2_REVIEW_PACKAGE/03_TARGET_DESIGN.md`

Diagram candidates: `../sqltoolsservice/refactor_docs/design_diagrams.tex`, `../sqltoolsservice/refactor_docs/STS2_REVIEW_PACKAGE/diagrams/STS2_DESIGN_VISUALS_V2.tex`

4 SQL Data Plane and Endpoint Adapters

Explain that product code imports SQL session/query semantics, not STS2 wire DTOs. The domain API is the contract for Query Studio, metadata, replay, and future hosted/web backends; STS2

JSON-RPC is the first binding.

4.1 Adapter Contract

Cover session identity, one-active-query semantics, ordered events, terminal completion, backpressure, cancellation, close semantics, capability honesty, data classification, and error normalization.

4.2 Backends

Describe the current STS2 JSON-RPC backend and fake backend. Reserve a subsection for future STS2 HTTP/WebSocket or hosted REST-style adapters, because the docs deliberately separate backend semantics from byte transport.

4.3 Settings and Operational Status

Include the user-facing switches `mssql.sqlDataPlane.enabled` and `mssql.sqlDataPlane.backend`, plus the data-plane status command. State that there are no separate `mssql.sts2.*` client settings.

Primary sources: `../coding-docs/ssms-query-docs/03-sts2-client-adapter-design.reviewed.md`, `../coding-docs/ssms-query-docs/query-studio-design-addendum.md`, `../vscode-mssql/extensions/mssql/src/services/sqlDataPlane/`, `../vscode-mssql/extensions/mssql/src/services/sts2/`

5 Metadata Substrate

5.1 CatalogModel, MetadataService, and MetadataStore

Document the three-layer stack:

- `CatalogModel`: pure immutable snapshots, structure-of-arrays storage, interned strings, section readiness, collation-aware name resolution.
- `MetadataService`: per-database engine, dedicated session, hydration H0–H7, generation bumping, DDL sniffing, digest polling, explicit refresh, lazy module-definition reads.
- `MetadataStore`: process singleton, server/database keys, leases, key-correct acquisition, warm idle TTL, LRU, server catalog, and shared consumer adapters.

5.2 Consumers and Readiness Honesty

Explain how Query Studio, native language service, AI completions, scripting, and Object Explorer v2 consume pinned snapshots or leases. Emphasize the two load-bearing rules: pin once per response, and failure is never rendered as emptiness.

5.3 Cache and Freshness

Cover the metadata cache codec/coordinator/manifest/store/freshness files, settings, offline mode, and what still needs real large-catalog data before cache and metadata perf gates are meaningful.

Primary sources: `../coding-docs/metadata-docs/metadata-substrate-design.md`, `../coding-docs/metadata-docs/README.md`, `../coding-docs/oe-docs/metadata_service_oe_v2_design.md`, `../vscode-mssql/extensions/mssql/src/services/metadata/`

Diagram candidates: `../coding-docs/metadata-docs/METADATA_DESIGN_VISUALS.tex`

6 Query Studio

6.1 Product Scope

Describe Query Studio as a custom text editor for `.sql`: embedded Monaco editor, toolbar, connection/database controls, execution/cancel/parse/plan commands, results/messages/plan tabs, reused grid work, status bar integration, AI inline completions, metadata-backed schema context, replay, and perftest hooks.

6.2 Host Architecture

Cover the document model and webview split:

- One `QueryStudioDocumentModel` per URI, with multiple panels attached to the same model.
- Text sync protocol, hashes, coalescing, resync safety valve, and custom-editor save/hot-exit behavior.
- `DocumentSessionBinding`, `ExecutionOrchestrator`, `RowStore`, result serializers, message log, status interop, and replay capture.
- Webview React shell, Monaco setup, toolbar, results layout, grid operations, and lazy tabs.

6.3 Execution and Result Semantics

Include batch splitting, selection/error-line mapping, SET wrappers for plans/parse, cancel and partial result truth, RowStore memory/spill privacy, compact row payloads, grid virtualization, XML/JSON links, NULL styling, filter/sort, timer, USE tracking, and transaction badge/guard.

6.4 Open Query Studio Decisions

Track plan tab rendering, grid convergence, Ctrl+R/keybinding parity, wide/blob perf scenarios, Save-As testing, and any SQLCMD/results-to-file/export scope decisions.

Primary sources: `../coding-docs/ssms-query-docs/04-query-studio-master-design.reviewed.md`, `../coding-docs/ssms-query-docs/query-studio-design-addendum.md`, `../coding-docs/ssms-query-doc`

grid-convergence-decision.md, ../vscode-mssql/extensions/mssql/src/queryStudio/, ../vscode-mssql/extensions/mssql/src/webviews/pages/QueryStudio/

Screenshot candidates: ../coding-docs/uxcaptures/, ../coding-docs/ssms-query-docs/mockup1.png, ../coding-docs/ssms-query-docs/mockup2.png

7 Native TypeScript T-SQL Language Service

7.1 Architecture

Document the native engine as an extension-host TypeScript service behind a feature router. The core should be described as rehostable and metadata-provider based: lexer, segmenter, sketch parser, overlay, binder, feature modules, provider adapters, scheduler, native engine, and STS v1 bridge.

7.2 Features

Cover non-AI completions, FK-aware joins, star expansion, INSERT/EXEC scaffolds, diagnostics with suppression ladder, hover, signature help, definition/peek, scripting emitters, document symbols, folding, and future semantic tokens/code actions.

7.3 Migration and Evidence

Explain the setting `mssql.queryStudio.languageService.engine`, native-vs-bridge routing, warm completion latency evidence, fourslash corpus, head-to-head scenario plans, and the eventual shadow-v1 retirement decision.

Primary sources: ../coding-docs/language-service-docs/05-tsql-language-service-design.md, ../coding-docs/language-service-docs/PROGRESS.md, ../vscode-mssql/extensions/mssql/src/sqlLanguage/, ../vscode-mssql/extensions/mssql/src/sqlScripting/

8 AI Completions, SDK Providers, and Replay

8.1 Schema Context and Inline Provider

Explain the ported schema-context pipeline: `catalogSchemaContextPayload`, `completionSchemaContextCore`, `completionSchemaContextService`, system-object catalog, deterministic rendering, budget profiles, and privacy boundaries around prompt material.

8.2 Model Providers

Document provider selection across GitHub Copilot, Anthropic, OpenAI, and xAI SDK providers; API key resolution; vendor allow lists; model catalogs; continuation model selection; and settings.

8.3 Debugging and Replay

Cover Inline Completion Debug, trace serialization/persistence/loading, replay cart, replay matrix, persisted Sessions tab, and how this connects to the broader feature-capture framework.

Primary sources: `../vscode-mssql/extensions/mssql/src/copilot/`, `../vscode-mssql/extensions/mssql/src/copilot/inlineCompletionDebug/`, `../vscode-mssql/extensions/mssql/src/diagnostics/featureCapture/`, `../coding-docs/settings.md`

9 Debug Console and In-Product Diagnostics

9.1 Diagnostics Core

Describe the event model, classification/redaction, sink routing, live-tail ring, Session Diag store, export bundle, rich collection, perf history providers, central upload, and self-test service.

9.2 Debug Console UI

Outline the pages: Overview, Consolidated Trace, Waterfall, Perf Test History, Session History, SQL Activity, Connections, Query & Results, Object Explorer, Completions, Exports, Settings, and Self-Test.

9.3 Replay and STS2 Stage Gates

Explain Stage A/B as extension-side diagnostics and shared renderers, Stage C as live STS2 source gated on STS2 preview tag and `IEnvelopeSink`, and Stage D as feature replay/replay-drive growth.

Primary sources: `../coding-docs/debug-docs/MSSQL_Debug_Console_Technical_Design.md`, `../coding-docs/debug-docs/MSSQL_Debug_Console_UX_Spec.md`, `../coding-docs/observability-docs/02-debug-console.md`, `../vscode-mssql/extensions/mssql/src/diagnostics/`, `../vscode-mssql/extensions/mssql/src/webviews/pages/DebugConsole/`

Screenshot candidates: `../coding-docs/debug-docs/screen1.png` through `../coding-docs/debug-docs/screen11.png`

10 Central Observability

10.1 One Store, Two Writers

Document the central SQL Server store as one shared schema with two ingestion paths: Debug Console uploads for ad-hoc sessions/imported runs, and `perftest push` for CI/local perf runs. Include the upload ledger, natural keys, content digest, upload policies, fixture conformance, and why files remain ground truth.

10.2 Privacy, Identity, Retention, and Readers

Cover upload preview, dropped/digested fields, no-secret rule, uploader identity, database roles, retention defaults, canned SQL views, Perf History central provider, Grafana, and future Bencher/OTLP projections.

Primary sources: `../coding-docs/observability-docs/central_observability_design.md`, `../coding-docs/observability-docs/options_for_central_tracing.md`, `../perftest/packages/perf-contracts/src/central/`, `../vscode-mssql/extensions/mssql/src/diagnostics/centralUpload.ts`

Diagram candidates: `../coding-docs/observability-docs/CENTRAL_OBSERVABILITY_VISUALS.tex`

11 perftest CLI and Harness

11.1 User Guide

This section should become the practical "what do I type" chapter. Pull heavily from the existing guide:

- `perftest doctor`, `scenarios list`, `run`, `report`, `history`, `trend`, `compare`, `diff`, `head-to-head`, `baseline`, `tag`, `cleanup`.
- Config files, run directories, reports, exit codes, official-vs-diagnostic metric rules, and environment-hash comparability.
- Central commands: `central init`, `central check`, `central health`, `central cleanup`, `central report`, `push`, and `push -dry-run`.

11.2 Implementation Design

Cover the CLI package, scenario engine, product driver extension, marker sink, result normalizer, SQLite store, diagnostic collectors, SQL provisioning, reports, central projection, and head-to-head report generation.

11.3 Scenarios and Gates

Document classic `query-10k-results`, Query Studio `querystudio-query-10k`, metadata/cache probes, central upload round-trip probes, SQL diagnostics, and maturity levels.

Primary sources: `../coding-docs/how_to_use_perftest.md`, `../perftest/docs/CLI.md`, `../perftest/docs/ARCHITECTURE.md`, `../perftest/docs/SCENARIO_AUTHORING.md`, `../perftest/docs/REGRESSION_MODEL.md`, `../perftest/docs/REPORTS.md`, `../perftest/PROGRESS.md`

Visual candidates: `../coding-docs/perftest-docs/PerfTest-Infographic.png`, `../coding-docs/perftest-docs/VS_Code_MSSQL_Performance_Harness.pdf`, `../perftest/grafana/central-observability-da`
`json`

12 Object Explorer v2

Object Explorer v2 is not in the user's minimum list, but it is a major branch consumer of the metadata substrate and appears in the `vscode-mssql` diff. Treat it as either a full section or an appendix.

12.1 Design Topics

Cover MetadataStore-backed server/database browsing, key-correct leases, readiness nodes, v2 tree controller, native commands, table preview, scripting handoff, classic handoff policy, settings, and preview/default-flip criteria.

Primary sources: `../coding-docs/oe-docs/metadata_service_oe_v2_design.md`, `../coding-docs/oe-docs/oe_view_design.md`, `../coding-docs/oe-docs/EXECUTION_PLAN.md`, `../vscode-mssql/extensions/mssql/src/objectExplorer/v2/`

13 Cross-Cutting Design Rules

The paper should keep these rules visible rather than burying them in individual chapters:

- New product features depend on SQL Data Plane semantics, not STS2 wire types.
- STS v1 is a temporary bridge or legacy handoff, not the substrate for new execution paths.
- Privacy is mechanical: classify fields, digest/drop at boundaries, never emit secrets, SQL text, rows, prompt text, or raw connection strings by default.
- Official metrics and diagnostic evidence stay separate.
- Readiness honesty: failed/loading/permission-denied/stale states are distinct from ready-empty.
- Contracts move through registry/generation/vendor-sync before producers emit new vocabulary.
- Replay traces and replayed spans must be identified so replay evidence never pollutes live evidence.

14 Screenshot and Figure Plan

14.1 Figures to Reuse from Existing TeX

- STS2 process architecture, event-capture seam, pump lifecycle, journal overlay, live tail, runtime config, and core state machines from `../sqltoolsservice/refactor_docs/design_diagrams.tex`.
- STS2 reviewed target design pages from `../sqltoolsservice/refactor_docs/STS2_REVIEW_PACKAGE/diagrams/STS2_DESIGN_VISUALS_V2.tex`.

- Metadata stack, identity/key-correctness, hydration/generation, readiness, and consumer diagrams from `../coding-docs/metadata-docs/METADATA_DESIGN_VISUALS.tex`.
- Central observability system landscape, ingest spine, policy preview, writer flows, and reader dashboards from `../coding-docs/observability-docs/CENTRAL_OBSERVABILITY_VISUALS.tex`.

14.2 Screenshots to Capture or Reuse

- Query Studio: editor shell, connection/database controls, execution toolbar, results grid, messages, plan handling, Replay Lab, language-service status, transaction badge, XML/JSON cell links, filter/sort, and wide/blob cases.
- Debug Console: overview, consolidated trace, waterfall, perf history, run analysis, completions page, exports/upload preview, self-test, SQL Activity, and settings.
- Inline Completion Debug: event grid, detail pane, toolbar, replay trace builder, replay cart, and Sessions tab.
- Object Explorer v2: server/database tree, readiness/failure nodes, table preview, script-as, legacy handoff confirmation.
- perftest: run `index.html`, report, trend, compare, head-to-head, central report, and Grafana dashboard.

15 Operator and User Workflows

This chapter should be concise but practical. Use `../coding-docs/settings.md` and `../coding-docs/how_to_use_perftest.md` as the starting point.

15.1 Suggested Workflows

- Turn on SQL Data Plane and Query Studio.
- Test native language service and inspect Language Service Status.
- Enable metadata cache, test warm acquire, and test offline mode.
- Switch Object Explorer to v2 preview and test table browse/preview/script commands.
- Enable AI completions, choose providers, capture traces, and replay completion requests.
- Open Debug Console, enable Session Diagnostics, elevate capture briefly, export, and upload to central.
- Run perftest locally, compare, generate head-to-head reports, push to central, and read the central report.

16 Validation Evidence and Test Matrix

Use one section to avoid scattering test claims across the paper. Group evidence by subsystem: STS2 unit/scenario/replay/invariant/mutation tests, Query Studio unit and perftest gates, metadata key-correctness/cache/freshness tests, language service fourslash and latency tests, AI completion replay tests, Debug Console/central upload tests, and perftest workspace tests.

17 Open Questions, Risks, and Possible Axe List

Keep this honest and easy to prune:

- STS2 M7 preview tag and capture effective-mode echo.
- Service-side `maxCellBytes` honoring.
- Query Studio plan tab rendering and grid convergence.
- Language service B14 default-flip and shadow-v1 retirement.
- Object Explorer v2 preview/default path and legacy handoff boundaries.
- Central observability hosting, ownership, retention, and support-bundle policy decisions.
- SDK provider shipping policy, API-key UX, and remote model privacy boundary.
- Hosted REST / web endpoint adapters and whether they are real product requirements or reserved architecture.
- MCP, DAB, and adjacent feature work: include only if the diff materially changes design or user workflows.

A Source Index

A.1 Cross-Repo State and Settings

- `../coding-docs/remaining_tasks.md`
- `../coding-docs/settings.md`
- `../coding-docs/observability-docs/07-phase2-query-studio-branch-guide.md`

A.2 STS2

- `../sqltoolsservice/docs/sts2/SPEC.md`
- `../sqltoolsservice/docs/sts2/CONTRACT.md`
- `../sqltoolsservice/docs/sts2/OBSERVABILITY.md`

- ../sqltoolsservice/docs/sts2/SCENARIO-MATRIX.md
- ../sqltoolsservice/refactor_docs/what_was_built.md
- ../sqltoolsservice/refactor_docs/STS2_REVIEW_PACKAGE/

A.3 Query Studio, Metadata, Language Service, and OE v2

- ../coding-docs/ssms-query-docs/04-query-studio-master-design.reviewed.md
- ../coding-docs/ssms-query-docs/03-sts2-client-adapter-design.reviewed.md
- ../coding-docs/ssms-query-docs/query-studio-design-addendum.md
- ../coding-docs/metadata-docs/metadata-substrate-design.md
- ../coding-docs/language-service-docs/05-tsql-language-service-design.md
- ../coding-docs/oe-docs/metadata_service_oe_v2_design.md

A.4 Diagnostics, Observability, and perftest

- ../coding-docs/debug-docs/MSSQL_Debug_Console_Technical_Design.md
- ../coding-docs/debug-docs/MSSQL_Debug_Console_UX_Spec.md
- ../coding-docs/observability-docs/central_observability_design.md
- ../coding-docs/how_to_use_perftest.md
- ../perftest/docs/CLI.md
- ../perftest/docs/ARCHITECTURE.md
- ../perftest/docs/REPORTS.md
- ../perftest/PROGRESS.md

B Initial Code Map

- ../vscode-mssql/extensions/mssql/src/services/sqlDataPlane/
- ../vscode-mssql/extensions/mssql/src/services/sts2/
- ../vscode-mssql/extensions/mssql/src/services/metadata/
- ../vscode-mssql/extensions/mssql/src/queryStudio/
- ../vscode-mssql/extensions/mssql/src/sqlLanguage/
- ../vscode-mssql/extensions/mssql/src/sqlScripting/
- ../vscode-mssql/extensions/mssql/src/copilot/

- `../vscode-mssql/extensions/mssql/src/diagnostics/`
- `../vscode-mssql/extensions/mssql/src/objectExplorer/v2/`
- `../sqltoolsservice/src/sts2/`
- `../sqltoolsservice/test/sts2/`
- `../perftest/packages/perftest-cli/`
- `../perftest/packages/perf-contracts/`